Quantum Neural Networks

Theory, Architectures, and Applications

1 Background

Classical machine learning has gained huge success in recent years especially deep learning models such as multi-layer perceptrons (MLP), Convolution Neural Networks (CNN), Recurrent Neural Netowrks (RNN), Transformers, and Graph Neural Netowrks (GNN). These architectures, due to its inductive bias and with development of hardware accelerators, have been applied in many fieds such as image processing, audio-speech processing, language modeling, and fraud detection. However, as the models are getting larger and larger, the computing power required is also growing exponentially. Governments and private companies are spending billions of dollar on building data center to satisfy the compute needs of now.

On the other hand, a new paradigm of computing started to shine on the stage – quantum computing.

2 Review of Classical Neural Networks

Before exploring Quantum Neural Networks (QNNs), it is essential to understand their classical counterparts. At their core, neural networks are parameterized mathematical models designed to approximate complex functions from data. Among the various architectures, the Multilayer Perceptron (MLP) is one of the most fundamental and widely studied. Despite its relatively simple structure, the MLP provides the theoretical foundation for understanding the expressive power of neural networks.

2.1 Architecture of Multilayer Perceptrons

An MLP is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer, as illustrated in Fig. 1. Each layer consists of multiple neurons, which perform affine transformations followed by nonlinear activation functions.

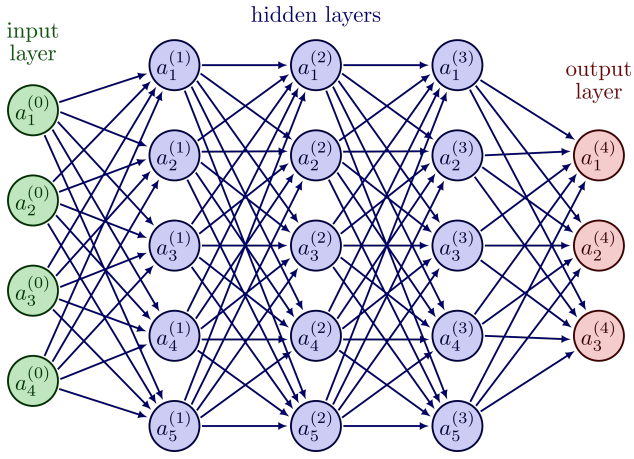


Figure 1: Architecture of a Multilayer Perceptron (MLP), consisting of an input layer, hidden layers, and an output layer.

Mathematically, the output of the l-th hidden layer can be expressed as

$$\mathbf{h}^{(l)} = \sigma\left(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

where $\mathbf{h}^{(0)} = \mathbf{x}$ denotes the input vector, $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the trainable weight matrix and bias vector, respectively, and σ is a nonlinear activation function. Common choices include Sigmoid, Tanh, and Rectified Linear Unit (ReLU).

By composing multiple layers, an MLP constructs a parameterized mapping from the input space to the output space. Let $A^{(l)}(\mathbf{h}) = W^{(l)}\mathbf{h} + \mathbf{b}^{(l)}$ denote the affine transformation at layer l. An MLP with L affine layers and a linear output layer can then be represented as

$$f_\theta(\mathbf{x}) = A^{(L)} \circ \sigma \circ A^{(L-1)} \circ \cdots \circ \sigma \circ A^{(1)}(\mathbf{x}) \tag{2.2}$$

where θ represents all trainable parameters and $\circ$ denotes function composition.

An important observation is that the nonlinear activation function is essential. Without nonlinearities, any composition of affine transformations remains an affine transformation, regardless of the number of layers. Therefore, nonlinear activations are what enable MLPs to represent functions beyond simple linear mappings.

2.2 MLPs as Function Approximators

The fundamental objective of an MLP is to approximate an unknown target function $f^*(\mathbf{x})$ using a parameterized function $f_\theta(\mathbf{x})$.

To understand how this works, consider a single-hidden-layer MLP with a scalar output:

$$f_\theta(\mathbf{x}) = \sum_{j=1}^{m} a_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j) + c \tag{2.3}$$

where m is the number of hidden neurons, $\mathbf{w}_j$ and b_j determine the transformation applied by the j-th neuron, and a_j controls its contribution to the final output.

This expression reveals a fundamental principle: an MLP approximates a complex function by combining multiple simpler nonlinear functions. Each hidden neuron generates a parameterized basis-like function, and the output layer constructs a weighted combination of these functions.

For example, consider the ReLU activation:

$$\sigma(x) = \max(0, x) \tag{2.4}$$

Each ReLU neuron introduces a piecewise-linear component. By combining sufficiently many such components, an MLP can construct increasingly complex piecewise-linear functions that approximate nonlinear target functions, as illustrated in Fig. 2.

Given a training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the network learns its parameters by minimizing a loss function. For example, in regression tasks, the Mean Squared Error (MSE) objective is

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\mathbf{x}_i) - y_i\right)^2 \tag{2.5}$$

In practice, the parameters are typically optimized through gradient-based methods using backpropagation.

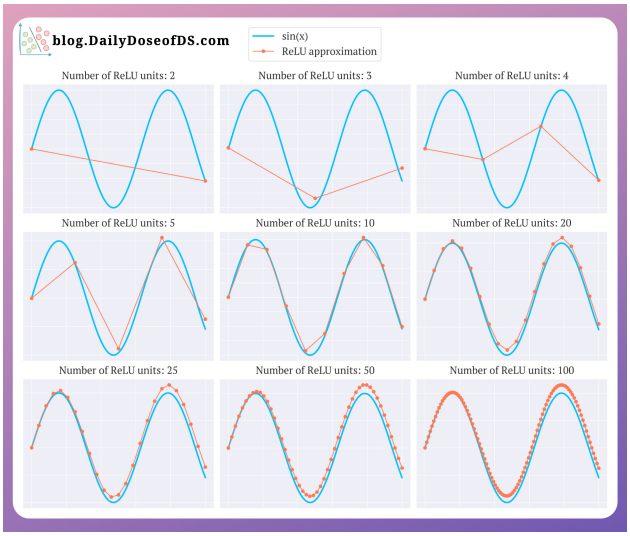


Figure 2: Illustration of function approximation using ReLU-based MLPs. Increasing the number of hidden neurons provides greater flexibility to approximate a nonlinear target function.

2.3 Universal Approximation Theorem

A natural question arises: how complex a function can an MLP approximate?

One of the most important theoretical results in neural network research is the Universal Approximation Theorem (UAT), established through a series of foundational works [**?**, **?**].

In particular, consider a continuous target function $f^* : K \to \mathbb{R}$, where $K \subset \mathbb{R}^d$ is compact. For any approximation tolerance $\epsilon > 0$, there exists a single-hidden-layer ReLU network with sufficiently many hidden neurons such that

$$\boxed{\sup_{\mathbf{x} \in K} |f^*(\mathbf{x}) - f_\theta(\mathbf{x})| < \epsilon} \tag{2.6}$$

In other words, even an MLP with only one hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy, provided that sufficient hidden neurons are available.

This theorem establishes the remarkable expressive potential of neural networks. However, several important distinctions must be made.

First, the theorem guarantees the *existence* of suitable parameters, but does not guarantee that a training algorithm can efficiently discover them. Second, it does not specify how many neurons or parameters are required to achieve a desired approximation accuracy. Finally, approximation capability does not necessarily imply good generalization to unseen data.

Although a single hidden layer is theoretically sufficient for universal approximation, deeper architectures may represent certain functions more efficiently by exploiting their compositional structure.

2.4 Beyond MLPs: Toward Quantum Neural Networks

Modern deep learning has extended the basic principles of MLPs into numerous specialized architectures. Convolutional Neural Networks (CNNs) exploit spatial structure, recurrent architectures such as RNNs, LSTMs, and GRUs model sequential dependencies, Transformers leverage attention mechanisms, and Graph Neural Networks (GNNs) incorporate graph structure. Despite their architectural differences, these models share the fundamental objective of learning parameterized functions from data.

This naturally leads to an intriguing question: can quantum computers also serve as function approximators?

While classical MLPs construct functions through compositions of affine transformations and nonlinear activations, variational QNNs construct functions through quantum data encoding, parameterized unitary transformations, and measurements. A typical QNN output can be expressed as

$$f_\theta(\mathbf{x}) = \langle 0|^{\otimes n} U^\dagger(\mathbf{x},\theta)\mathcal{O}U(\mathbf{x},\theta)\,|0\rangle^{\otimes n} \tag{2.7}$$

where $U(\mathbf{x},\theta)$ represents the parameterized quantum circuit and $\mathcal{O}$ denotes a measurement observable.

Thus, both MLPs and QNNs can be viewed as parameterized function approximators, yet they rely on fundamentally different computational mechanisms. This raises several important questions: How do QNNs represent functions? What determines their expressive power? And how does their function approximation capability compare with that of classical MLPs?

These questions motivate the following investigation into the mathematical representation and expressive properties of Quantum Neural Networks.

3 Quantum Neural Networks

3.1 Basics of Quantum Computing

If we have multiple qubits, let's say N qubits. Each qubit, individually is operated by a quantum gate $U_1, U_2, \cdots, U_N$. Then the operation matrix on the entire quantum state is

$$U = U_1 \otimes U_2 \otimes U_3 \otimes \cdots \otimes U_N$$

CNOT gate is an interesting gate that takes 2 qubits as input. Its main purpose is to produce conditional independence between these 2 qubits. In these 2 qubits, one is target and the other is control. The logic is:

$$|c,t\rangle \xrightarrow{\text{CNOT}} |c, t \oplus c\rangle$$

That is flipping the target qubit if the control qubit is 1, otherwise do nothing. For example,

if we let q_0 be the target and q_1 be the control and we use little-endian format $|q_1 q_0\rangle$.

$$|00\rangle \to |00\rangle$$
$$|01\rangle \to |01\rangle$$
$$|10\rangle \to |11\rangle$$
$$|11\rangle \to |10\rangle$$

The operation matrix of CNOT gate is therefore:

$$CNOT_{q_1 \to q_0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

For example,

$$CNOT_{q_1 \to q_0} |10\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle$$

Please be aware that if q_0 is the control and q_1 is the target, then

$$CNOT_{q_0 \to q_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

For example,

$$CNOT_{q_0 \to q_1} |01\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle$$

Then, how is entanglment produced using CNOT gate? In Figure 3, we have two qubits, both in $|0\rangle$ as initial state. Then, one of them undergo Hadmard operation, which makes the state become:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |01\rangle)$$

After applying a CNOT gate on these 2 qubits, we have:

$$CNOT_{q_0 \to q_1} |\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Now, these 2 qubits are entangled! This means that if we know the state of one of the qubits, we immediately know the state of the other without measuring it.

Rule of thumb. For a two-qubit syste:

$$|\psi_1 \psi_0\rangle = a\,|00\rangle + b\,|01\rangle + c\,|10\rangle + |11\rangle$$

The quickest way to check if these two qubits are entangled is:

$$\begin{cases} ad - bc \neq 0, & \text{entangled} \\ ad - bc = 0, & \text{product state/non-entangled} \end{cases} \tag{3.1}$$

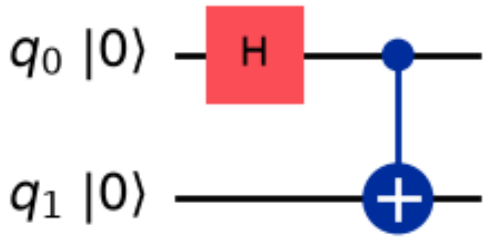


Figure 3: How entanglment is produced in bell state

3.2 Dissection of QNN

Similar to classical neural networks, quantum neural networks (QNN) also tries to universally approximate functions but in a foundamental different way. Figure 4 shows a **3-qubit** circuit architecture of a standard quantum neural networks. Simply speaking, it is composed of layers of rotation gate (X,Y,Z) and CNOT gate. In the end, the each wire has a measurement gate (usually Z-basis). Let's look at each gate one by one. Firstly, there are three types of

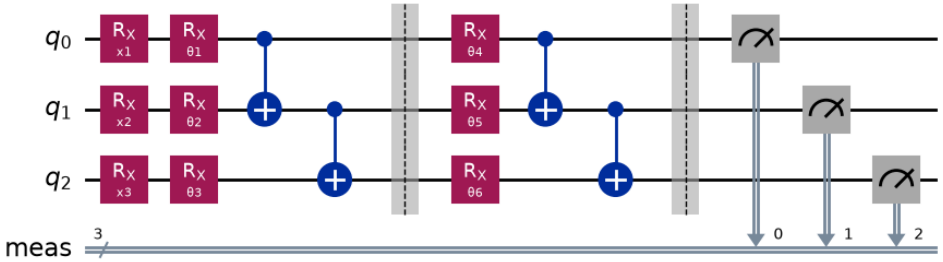


Figure 4: A standard quantum neural network

rotation gate R_x, R_y, R_z. Any single-qubit gate is a 2×2 unitary matrix and these three matrices, variants of Pauli-matrices, rotates the vector in the bloch sphere about x, y, z axis by θ degrees. Their mathmatical definition is as follows:

$$R_x(\theta) \equiv e^{-i\theta X/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})X = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}. \tag{3.2}$$

$$R_y(\theta) \equiv e^{-i\theta Y/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})Y = \begin{bmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}. \tag{3.3}$$

$$R_z(\theta) \equiv e^{-i\theta Z/2} = \cos(\frac{\theta}{2})I - i\sin(\frac{\theta}{2})Z = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}. \tag{3.4}$$

Controlled-NOT (CNOT) gate has two input qubits, known as control qubit and target qubit, respectively. In terms of the computational basis, the action of CNOT is given by

$|c\rangle |t\rangle \rightarrow |c\rangle |t \oplus c\rangle$. That is if the control qubit is set to $|1\rangle$ then the target qubit is flipped, otherwise the target qubit is left alone. Thus, in the computational basis $|contro, target\rangle$ the matrix representation of CNOT is:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

For example, if a two-qubit system is in state $|11\rangle$, which in basis vector is $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$. Here we follow little-endian (LSB)convention ($|q_1 q_0\rangle$) and let $|q_0\rangle$ be the control qubit and $|q_1\rangle$ be the target qubit. When a CNOT gate is applied to it:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle$$

$$CNOT\,|11\rangle \rightarrow |01\rangle$$

These three rotation gates and CNOT gate are also known universal gates in quantum computer, similar to NOR and NAND in classical computer. Any unitary matrix can be decomposed to sequenec of rotation gates and CNOT gate. Similar to classical neural network, quantum neural network also has input, hidden and output layers, which corresponds to encoding, rotation and entanglment in Hilbert Space, and measurement. Let's look at them one by one.

3.3 Input Layer: Encoding

For most of machine learning tasks, the data is classical. Therefore, we need a way to encode the classical data into quantum states. Three main encoding approaches are:

1. **Amplitude Encoding**
2. **Basis Encoding**
3. **Angular Encoding**

Of course, there are other more advanced encoding such as Pauli Encoding and IQP Encoding but it's not in the scope of this article. Here, we first talk about the most common encoding method - **Angular Encoding** because it's the easiest to prepare from engineering perspective. Just as the name suggests, the classical data x_i is mapped into a rotation angle using rotation gates. Figure 5 shows how a 3-dimension input vector is embedded into a quantum circuit. Each qubit, represented by a single line here, starts with 0 basis state $|0\rangle$. Then qubit is followed by a rotation gate (can be R_x, R_y, R_z). These rotation gates will then be parametrized

by the input values x_1, x_2, x_3. For example, for the first qubit (little-endian), the starting state is $|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}$. Then the rotation gate, let's say R_x, is parametrized by x_1:

$$R_x(x_1) = \begin{bmatrix}\cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2})\end{bmatrix}.$$

Then the qubit is operated as:

$$R_x(x_1)\,|0\rangle = \begin{bmatrix}\cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2})\end{bmatrix} \times \begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}\cos(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2})\end{bmatrix}$$

This is equivalent to:

$$|0\rangle \rightarrow \cos(\frac{x_1}{2})\,|0\rangle + (-i\sin(\frac{x_1}{2}))\,|1\rangle$$

If you use R_y, then you will have:

$$R_y(x_1)\,|0\rangle = \begin{bmatrix}\cos(\frac{x_1}{2}) & -\sin(\frac{x_1}{2})\\ \sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2})\end{bmatrix} \times \begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}\cos(\frac{x_1}{2})\\ \sin(\frac{x_1}{2})\end{bmatrix}$$

Similarly, for the rest of input values x_2, x_3, x_4, the quantum states of the qubits become (assume we use R_x):

$$q_1(x_2) = \begin{bmatrix}\cos(\frac{x_2}{2})\\ -i\sin(\frac{x_2}{2})\end{bmatrix}, \quad q_2(x_3) = \begin{bmatrix}\cos(\frac{x_3}{2})\\ -i\sin(\frac{x_3}{2})\end{bmatrix}$$

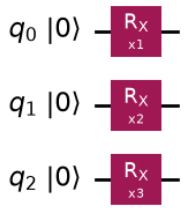


Figure 5: Input layer of a QNN using angluar encoding.

If we look at the 3-qubit system holistically, which is also known as product state, the state transitions from $|000\rangle$ to $|q_2q_1q_0\rangle$. Mathmatically as follows:

$$\begin{bmatrix}1\\0\\0\\0\\0\\0\\0\\0\end{bmatrix} \xrightarrow{\text{angle encoding}} \begin{bmatrix}\cos(\frac{x_3}{2})\\ -i\sin(\frac{x_3}{2})\end{bmatrix} \otimes \begin{bmatrix}\cos(\frac{x_2}{2})\\ -i\sin(\frac{x_2}{2})\end{bmatrix} \otimes \begin{bmatrix}\cos(\frac{x_1}{2})\\ -i\sin(\frac{x_1}{2})\end{bmatrix}$$

This is equivalent to:

$$|q_2 q_1 q_0\rangle = |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$$

The statevector of entire circuit can be rewritten into tensor product of statevector of each single qubit becuase entanglement has not yet formed.
The tensor product of three 2×1 vectors is a 8×1 vector, which is expanded as follows:

$$\psi(x_3, x_2, x_1) = \begin{bmatrix} \cos(\frac{x_3}{2})\cos(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ -i\cos(\frac{x_3}{2})\cos(\frac{x_2}{2})\sin(\frac{x_1}{2}) \\ -i\cos(\frac{x_3}{2})\sin(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ -\cos(\frac{x_3}{2})\sin(\frac{x_2}{2})\sin(\frac{x_1}{2}) \\ -i\sin(\frac{x_3}{2})\cos(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ -\sin(\frac{x_3}{2})\cos(\frac{x_2}{2})\sin(\frac{x_1}{2}) \\ -\sin(\frac{x_3}{2})\sin(\frac{x_2}{2})\cos(\frac{x_1}{2}) \\ i\sin(\frac{x_3}{2})\sin(\frac{x_2}{2})\sin(\frac{x_1}{2}) \end{bmatrix}$$

If we look each qubit individually, the encoding is a 2×2 matrix. However, if we look the encoding holistically on the entire 3 qubit system, it's a $2^3 \times 2^3 = 8 \times 8$ matrix, which is also a tensor product of all three rotation matrices:

$$\begin{bmatrix} \cos(\frac{x_3}{2}) & -i\sin(\frac{x_3}{2}) \\ -i\sin(\frac{x_3}{2}) & \cos(\frac{x_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{x_2}{2}) & -i\sin(\frac{x_2}{2}) \\ -i\sin(\frac{x_2}{2}) & \cos(\frac{x_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{x_1}{2}) & -i\sin(\frac{x_1}{2}) \\ -i\sin(\frac{x_1}{2}) & \cos(\frac{x_1}{2}) \end{bmatrix}$$

This makes sense becuase with 3 qubits, the quantum state is in $\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^8$ and the operation shall also be a 8×8 matrix as follows. To make it readable, let's define:

$$c_j = \cos\left(\frac{x_j}{2}\right), \qquad s_j = \sin\left(\frac{x_j}{2}\right), \qquad j = 1, 2, 3, 4.$$

$$U = R_x(x_3) \otimes R_x(x_2) \otimes R_x(x_1)$$

The matrix is as follows:

$$\begin{bmatrix} c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 \\ -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 \\ -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 \\ -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 \\ -is_3c_2c_1 & -s_3c_2s_1 & -s_3s_2c_1 & is_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 \\ -s_3c_2s_1 & -is_3c_2c_1 & is_3s_2s_1 & -s_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 \\ -s_3s_2c_1 & is_3s_2s_1 & -is_3c_2c_1 & -s_3c_2s_1 & -ic_3s_2c_1 & -c_3s_2s_1 & c_3c_2c_1 & -ic_3c_2s_1 \\ is_3s_2s_1 & -s_3s_2c_1 & -s_3c_2s_1 & -is_3c_2c_1 & -c_3s_2s_1 & -ic_3s_2c_1 & -ic_3c_2s_1 & c_3c_2c_1 \end{bmatrix} \tag{3.5}$$

Please be aware that since we use little-endian convention, the tensor product of operation matrix follows the same order as the qubit. If our initial state is $|0\rangle$, then the resulting state is the first column of this matrix. In fact, the matrix is

$$U_{\mathbf{b},\mathbf{b}'} = (-i)^{d_H(\mathbf{b},\mathbf{b}')} \prod_{j:b_j=b'_j} c_j \prod_{j:b_j\neq b'_j} s_j$$

where $d_H(\mathbf{b}, \mathbf{b}')$ is Hamming distance between target state and initial state.
Before concluding angle encoding, there is one more thing - **scaling**. As you might have observed, the trignometric functions are all periodic (2π) meaning that

$$|\psi(x)\rangle = |\psi(x + 2\pi)\rangle$$

If the range of classical data is larger than 2π, then two different classical values will be mapped to the same quantum state, causing information loss. Therefore, scaling must be conducted such that the input values, before encoding, must be between 0 and π. This is becuase the periodicity of cosine and sine functions are 2π, so the periodicity in $\cos(\frac{x}{2})$ is π. A simple and most common way to do scaling is as follows:

$$\theta = \pi \frac{x - x_{min}}{x_{max} - x_{min}}$$

Now, the features are mapped into a non-repeatitive and continuous angle space. Finally, there are many variants of angle encoding such as XYZ encoding, data-reuploading, multiscale, and adaptive encoding. What has to be aware of is that encoding can have notrivial impact on the representation capacity of the model.

3.4 Hidden Layer - Rotation

Now, after angle encoding, each qubit is in the state

$$|\psi\rangle \cos(\frac{x_i}{2}) |0\rangle + (-i \sin(\frac{x_i}{2})) |1\rangle = \begin{bmatrix} \cos(\frac{x_1}{2}) \\ -i \sin(\frac{x_1}{2}) \end{bmatrix}.$$

Then, this qubit will go through another rotation gate, let's say R_x, which is parametrized by θ, resembling the weight in classical neural networks. The state becomes:

$$R_x(\theta) |\psi\rangle = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i \sin(\frac{\theta}{2}) \\ -i \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix} \times \begin{bmatrix} \cos(\frac{x_i}{2}) \\ -i \sin(\frac{x_i}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - \sin(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \\ -i \sin(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - i \cos(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \end{bmatrix}$$

Using following trignometric identity, we can further simplify.

$$\cos(A + B) = \cos(A) \cos(B) - \sin(A) \sin(B)$$
$$\sin(A + B) = \sin(A) \cos(B) + \cos(A) \sin(B)$$

$$\begin{bmatrix} \cos(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - \sin(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \\ -i \sin(\frac{\theta}{2}) \cos(\frac{x_i}{2}) - i \cos(\frac{\theta}{2}) \sin(\frac{x_i}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta + x_i}{2}) \\ -i \sin(\frac{\theta + x_i}{2}) \end{bmatrix}$$

For example, in Figure 6, the first qubit becomes:

$$|q_0\rangle = \begin{bmatrix} \cos(\frac{\theta_1 + x_1}{2}) \\ -i \sin(\frac{\theta_1 + x_1}{2}) \end{bmatrix}.$$

For the rest of qubits, the states becomes:

$$q_1 = \begin{bmatrix} \cos(\frac{\theta_2 + x_2}{2}) \\ -i \sin(\frac{\theta_2 + x_2}{2}) \end{bmatrix}, \quad q_2 = \begin{bmatrix} \cos(\frac{\theta_3 + x_3}{2}) \\ -i \sin(\frac{\theta_3 + x_3}{2}) \end{bmatrix}$$

The entire 3-qubit system then becomes the tensor product of these four vectors:

$$\psi_{q_3q_2q_1} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_2+x_2}{2}) \\ -i\sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix}$$

Which can be expaned as:

$$\begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.6}$$

Similarily, the operation matrix on these 3-qubits system is also the tensor product of these three 2×2 matrices:

$$U_{hidden} = \begin{bmatrix} \cos(\frac{\theta_3}{2}) & -i\sin(\frac{\theta_3}{2}) \\ -i\sin(\frac{\theta_3}{2}) & \cos(\frac{\theta_3}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_2}{2}) & -i\sin(\frac{\theta_2}{2}) \\ -i\sin(\frac{\theta_2}{2}) & \cos(\frac{\theta_2}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_1}{2}) & -i\sin(\frac{\theta_1}{2}) \\ -i\sin(\frac{\theta_1}{2}) & \cos(\frac{\theta_1}{2}) \end{bmatrix}$$

Similarily, if we define:

$$c_j = \cos(\frac{\theta_j}{2}), \quad s_j = \sin(\frac{\theta_j}{2}), \quad j = 1, 2, 3, 4$$

Then, this 8×8 matrix is the same as Eq. (3.5)

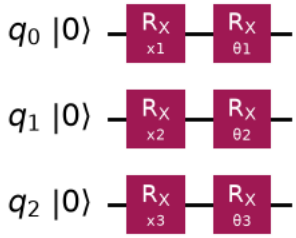


Figure 6: First hidden layer of rotation gates

3.5 Hidden Layer - Entanglement

What's getting more interesting is the CNOT gate in the next layer. If we only apply rotation gates on each qubit separately, there is no interactions between qubits. CNOT gate allows

interactions between qubits so that information can be passed across qubits instead of just staying at only a single qubit. A good analogy is that in classical neural networks, each neuron takes all the neurons in the previous layer as inputs, if we assume fully-connected layers.
Although we have a 3-qubit system, but let's look at the first two qubits q_0, q_1. In Figure 7, after encoding layer and first rotation layer, the state of each qubit is:

$$\begin{aligned} |q_0\rangle &= \cos(\frac{\theta_1 + x_1}{2}) |0\rangle + (-i \sin(\frac{\theta_1 + x_1}{2})) |1\rangle \\ |q_1\rangle &= \cos(\frac{\theta_2 + x_2}{2}) |0\rangle + (-i \sin(\frac{\theta_2 + x_2}{2})) |1\rangle \end{aligned} \tag{3.7}$$

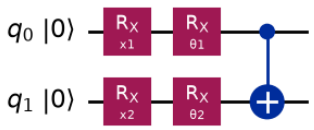


Figure 7: CNOT gate layer on two qubits after rotation gates

The state of entire system is:

$$|q_1 q_0\rangle = \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} \tag{3.8}$$

If we apply a CNOT gate on these two qubits, then we have:

$$CNOT\,|q_1 q_0\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_1+x_1}{2}) \cos(\frac{\theta_2+x_2}{2}) \\ -\sin(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_1+x_1}{2}) \sin(\frac{\theta_2+x_2}{2}) \\ -i \cos(\frac{\theta_2+x_2}{2}) \sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.9}$$

As you might observe, the second element and the fourth element of state vector got swapped. Now, these two qubits are entangled! This means that the system cannot be rewriten into a tensor product of two single qubit anymore. You can easily verify this by yourself using Eq. 3.1.

$$CNOT\,|q_1 q_0\rangle \neq |q_1\rangle \otimes |q_0\rangle$$

Now, if take the third qubit into consideration and look at the entire system, the operation matrix is:

$$U_{entangle} = I \otimes CNOT_{q_0 \to q_1} \tag{3.10}$$

where I is 2×2 identity matrix. To expand, we have

$$
\begin{aligned}
U_{entangle} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}
\end{aligned}
\tag{3.11}
$$

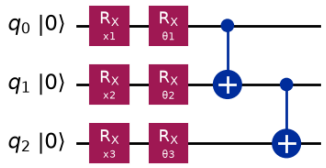


Figure 8: Entanglement layer in QNN

Then, we can apply this matrix 3.11 on the statevector we derived from Eq 3.6:

$$
\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix}
\tag{3.12}
$$

Please notice that the original order $(1, 2, 3, 4, \quad 5, 6, 7, 8)$ becomes $(1, 4, 3, 2, \quad 5, 8, 7, 6)$. For every "subgroup", the second and fourth element are swapped.
Then, as Figure 8 shows, we apply CNOT gate on the second and third qubits q_1, q_2, the

operation matrix is:

$$U_{entangle} = CNOT_{q_1 \to q_2} \otimes I$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Then, we apply this matrix on the statevector:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} = \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.13}$$

If you look carefully, What happens is that compare with Eq. 3.12, swapping happens as follows:

$$3 \leftrightarrow 7, \quad 4 \leftrightarrow 8$$

As you might observe, you can apply a CNOT gate on any of the two qubits (e.g., q_0, q_2) and CNOT gates are essentially just permutation matrix that induces a permutation of amplitude in statevector. The magic is that all these four qubits are entangled meaning the statevector is not simply a tensor product of four single-qubit statevector:

$$|q_2 q_1 q_0\rangle \neq |q_2\rangle \otimes |q_1\rangle \otimes |q_0\rangle$$

3.6 Information Propogation through CNOT Gates

The importance of CNOT gate is allowing information be propogatied from control qubit to target qubit, if we measure in Z-basis. Let's look at a 2-qubit system first. Previously, we have derived states of 2-qubit system in Eq. 3.8 that after a layer of encoding and rotation.

The probability of observing $q_1 = 0$ is as follows.

$$\begin{aligned} \text{Let} \quad \alpha = \frac{x_1 + \theta_1}{2}, \quad \beta = \frac{x_2 + \theta_2}{2} \\ P(|q_1\rangle = |0\rangle) &= \cos^2(\alpha)\cos^2(\beta) + \cos^2(\beta)\sin^2(\alpha) \\ &= \cos^2(\beta)(\cos^2(\alpha) + \sin^2(\alpha)) \\ &= \cos^2(\beta) = \cos(\frac{x_2 + \theta_2}{2}) \end{aligned} \tag{3.14}$$

As you can see, the probability of each state in q_1 is a function of $x2$ and θ_2:

$$P(|q_1\rangle = |0\rangle) = f(x_2, \theta_2)$$

However, after a CNOT gate, the statevector becomes Eq. 3.9. The The probability of observing $q_1 = 0$ becomes as follwoing.

$$\begin{aligned} P(|q_1\rangle = |0\rangle) &= \cos^2(\alpha)\cos^2(\alpha) + \sin^2(\alpha)\sin^2(\beta) \\ &= \cos^2(\frac{x_1 + \theta_1}{2})\cos^2(\frac{x_2 + \theta_2}{2}) + \sin^2(\frac{x_1 + \theta_1}{2})\sin^2(\frac{x_2 + \theta_2}{2}) \\ &= f(x_1, x_2, \theta_1, \theta_2) \end{aligned} \tag{3.15}$$

Now, as you can see, the probability amplitude of q_1 becomes a function of $x_1, x_2, \theta_1, \theta_2$! The information from q_0, the first qubit, both input value x_1 and parameters θ_1 are passed down to q_1. This gives the circuits much greater expressivity. However, another thing to notice is that the probability of q_0 does not change before or after CNOT gate.

$$P(|q_0\rangle = |0\rangle) = \cos^2(\frac{\theta_1 + x_1}{2}) \tag{3.16}$$

Intrestingly, the infomration did not pass from target to control. However, this is given we measure in Z-basis. Measurement in other basis will lead to different sitautions.
Again, if we compare with classical neural networks, it's a function of all the input values as Eq. 3.17 shows. This is also knonw as fully connected layers. If the output of a classical Nueral Networks is function of only a single input x_i, it will not be able learn the latent structure of the data, significantly harming the expressivity. The entanglement layer tries to do the similar thing.

$$\begin{aligned} f_{NN} &= f(\sigma(W^T X + b)) \\ &= f(\sigma(\sum_{n=1}^{N} w_n x_n + b)) \\ &= f(x_1, x_2, \cdots, x_N, W) \end{aligned} \tag{3.17}$$

Now, let's look at the 3-qubit system. Before the entanglment layer, the probability of observing the fourth qubit $|q_2\rangle = |0\rangle$ can be derived from Eq. 3.6 as follows:

$$\begin{aligned}
\text{Let} \alpha_j &= \frac{x_j + \theta_j}{2}, \quad j = 1, 2, 3 \\
P(q_2 = 0) &= |\psi_{000}|^2 + |\psi_{001}|^2 + |\psi_{010}|^2 + |\psi_{011}|^2 \\
&= \cos^2(\alpha_3)\cos^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\cos^2(\alpha_2)\sin^2(\alpha_1) \\
&+ \cos^2(\alpha_3)\sin^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\sin^2(\alpha_2)\sin^2(\alpha_1) \\
&= \cos^2(\alpha_3)[\cos^2(\alpha_2)(\cos^2(\alpha_1) + \sin^2(\alpha_1)) + \sin^2(\alpha_2)(\cos^2(\alpha_1) + \sin^2(\alpha_1))] \\
&= \cos^2(\alpha_3) \\
&= \cos^2(\frac{\theta_3 + x_3}{2})
\end{aligned} \tag{3.18}$$

As you can see, without CNOT (entanglement) layer, regardless how many rotation gate you add to the third qubit q_2 (this also applied to other qubits), the probability amplitude for each qubit is just a function of its encoding input x_3 and the parameters of the rotation gate in its own path θ_3. Mathmatically, it's $P(q_2 = 0) = f(\theta_3, x_3)$ The information will never be propagated across the lines. However, after the first entanglement layer, according to Eq. 3.13, the the probability amplitude of $|q_3\rangle = |0\rangle$ becomes as follows:

$$\begin{aligned}
P(q_2 = 0) &= \cos^2(\alpha_3)\cos^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_3)\sin^2(\alpha_2)\sin^2(\alpha_1) \\
&+ \sin^2(\alpha_3)\sin^2(\alpha_2)\cos^2(\alpha_1) + \sin^2(\alpha_3)\cos^2(\alpha_2)\sin^2(\alpha_1) \\
&= \cos^2(\alpha_3)[\cos^2(\alpha_2)\cos^2(\alpha_1) + \sin^2(\alpha_2)\sin^2(\alpha_1)] \\
&+ \sin^2(\alpha_3)[\sin^2(\alpha_2)\cos^2(\alpha_1) + \cos^2(\alpha_2)\sin^2(\alpha_1)] \\
&= \frac{\cos^2(\alpha_3)}{2}[1 + \cos(2\alpha_2)\cos(2\alpha_1)] + \frac{\cos^2(\alpha_3)}{2}[1 - \cos(2\alpha_2)\cos(2\alpha_1)] \\
&= \frac{1}{2}[1 + \cos(2\alpha_3)\cos(2\alpha_2)\cos(2\alpha_1)] \\
&= \frac{1}{2}[1 + \cos(\theta_3 + x_3)\cos(\theta_2 + x_2)\cos(\theta_1 + x_1)] \\
&= \frac{1}{2}[1 + \prod_{j=1}^{3}\cos(\theta_j + x_j)]
\end{aligned} \tag{3.19}$$

If we look back at, it's clear that

$$\begin{aligned}
P_{\text{before}}(q_2 = 0) &= \cos^2(\frac{\theta_3 + x_3}{2}) \\
P_{\text{after}}(q_2 = 0) &= \frac{1}{2}[1 + \prod_{j=1}^{3}\cos(\theta_j + x_j)]
\end{aligned} \tag{3.20}$$

Now, you can clearly see what magic the entanglment layer has done. If you measure q_3 before entanglment, its probability amplitude is determined by only θ_4, x_4 whereas after entanglment it's determined by all the inputs and parameters. As said before, after entanglment, the statevector cannot be rewritten into tensor product of statevector of each single qubit.

Therefore, all the computation after this point will be holistic, meaning that we need to do $(2^n \times 2^n) \times (2^n \times 1)$ matrix-vector calculation at each layer, which is a big challenge for classical computer. Imagine if you have 30 qubits, the dimension is 1,073,741,924. None of the CPUs, GPUs, or TPUs has enough RAM to do this calculation. However, it can be easily done by a qunatum computer, in fact IBM machines such as Nighthawk or Heron family offer 120 qubits and this number will be increasing in the next couple of years. In terms of large matrix calculation, the quantum advantage is clear.
Alright, let's keep watching how this quantum neural network evolve. As Figure 9 shows, another rotation layer is applied after entanglement layer, as the second hidden layer. The rotation gates are parametrized as $\theta_4, \theta_5, \theta_6$ respectively. The operation matrix of this layer

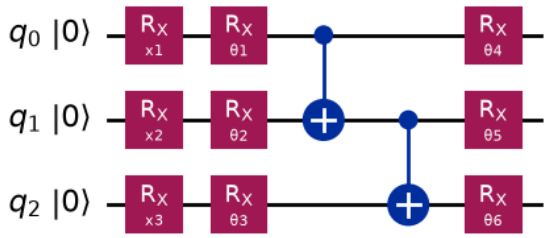


Figure 9: Quantum Neural Network with 2 hidden layers

is then:

$$
\begin{aligned}
U_{hidden} &= R_x(\theta_6) \otimes R_x(\theta_5) \otimes R_x(\theta_4) \\
&= \begin{bmatrix} \cos(\frac{\theta_6}{2}) & -i\sin(\frac{\theta_6}{2}) \\ -i\sin(\frac{\theta_6}{2}) & \cos(\frac{\theta_6}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_5}{2}) & -i\sin(\frac{\theta_5}{2}) \\ -i\sin(\frac{\theta_5}{2}) & \cos(\frac{\theta_5}{2}) \end{bmatrix} \otimes \begin{bmatrix} \cos(\frac{\theta_4}{2}) & -i\sin(\frac{\theta_4}{2}) \\ -i\sin(\frac{\theta_4}{2}) & \cos(\frac{\theta_4}{2}) \end{bmatrix} \\
&= \begin{bmatrix}
c_6c_5c_4 & -ic_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 \\
-ic_6c_5s_4 & c_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 \\
-ic_6s_5c_4 & -c_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 \\
-c_6s_5s_4 & -ic_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 \\
-is_6c_5c_4 & -s_6c_5s_4 & -s_6s_5c_4 & is_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 \\
-s_6c_5s_4 & -is_6c_5c_4 & is_6s_5s_4 & -s_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 \\
-s_6s_5c_4 & is_6s_5s_4 & -is_6c_5c_4 & -s_6c_5s_4 & -ic_6s_5c_4 & -c_6s_5s_4 & c_6c_5c_4 & -ic_6c_5s_4 \\
is_6s_5s_4 & -s_6s_5c_4 & -s_6c_5s_4 & -is_6c_5c_4 & -c_6s_5s_4 & -ic_6s_5c_4 & -ic_6c_5s_4 & c_6c_5c_4
\end{bmatrix}
\end{aligned}
\tag{3.21}
$$

The new statevector becomes:

$$U_{hidden} \times \begin{bmatrix} \cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\sin(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ i\sin(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\sin(\frac{\theta_2+x_2}{2})\cos(\frac{\theta_1+x_1}{2}) \\ -i\cos(\frac{\theta_3+x_3}{2})\cos(\frac{\theta_2+x_2}{2})\sin(\frac{\theta_1+x_1}{2}) \end{bmatrix} \tag{3.22}$$

Firstly, let's define:

$$\begin{aligned} C_1 &= \cos(\frac{\theta_1+x_1}{2}), \quad S_1 = \sin(\frac{\theta_1+x_1}{2}) \\ C_2 &= \cos(\frac{\theta_2+x_2}{2}), \quad S_2 = \sin(\frac{\theta_2+x_2}{2}) \\ C_3 &= \cos(\frac{\theta_3+\theta_6+x_3}{2}), \quad S_3 = \sin(\frac{\theta_3+\theta_6+x_3}{2}) \\ c_j &= \cos(\frac{\theta_j}{2}), \quad s_j = \sin(\frac{\theta_j}{2}) \end{aligned}$$

$$|q_2q_1q_0\rangle = \begin{bmatrix} C_3c_5(c_4C_2C_1 + is_4S_2S_1) + S_3s_5(ic_4S_2C_1 + s_4C_2S_1) \\ C_3c_5(-is_4C_2C_1 - c_4S_2S_1) + S_3s_5(s_4S_2C_1 + ic_4C_2S_1) \\ C_3s_5(-ic_4C_2C_1 + s_4S_2S_1) + S_3c_5(-c_4S_2C_1 + is_4C_2S_1) \\ C_3s_5(-s_4C_2C_1 + ic_4S_2S_1) + S_3c_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3c_5(-ic_4C_2C_1 + s_4S_2S_1) + C_3s_5(-c_4S_2C_1 + is_4C_2S_1) \\ S_3c_5(-s_4C_2C_1 + ic_4S_2S_1) + C_3s_5(is_4S_2C_1 - c_4C_2S_1) \\ S_3s_5(-c_4C_2C_1 - is_4S_2S_1) + C_3c_5(-ic_4S_2C_1 - s_4C_2S_1) \\ S_3s_5(is_4C_2C_1 + c_4S_2S_1) + C_3c_5(-s_4S_2C_1 - ic_4C_2S_1) \end{bmatrix} \tag{3.23}$$

However, as the circuit depth increases, manually deriving the statevector quickly becomes computationally intractable. This naturally raises an important question: can we abstract these increasingly complex expressions into a more compact mathematical representation that allows us to analyze the QNN systematically?

Fortunately, the answer is yes! In fact, under the data-encoding schemes considered here, the output of a QNN can be represented as a **finite Fourier series**.

To understand why, let us revisit Eq. 3.23. A closer examination reveals that each probability amplitude consists of combinations of sine and cosine functions. By applying standard trigonometric identities, these expressions can be rewritten in the general form

$$a_j(x,\theta) = \sum_{\alpha\in\Lambda} [A_{j,\alpha}(\theta)\cos(\alpha x) + B_{j,\alpha}(\theta)\sin(\alpha x)] \tag{3.24}$$

where Λ denotes a finite set of frequencies, and $A_{j,\alpha}(\theta)$ and $B_{j,\alpha}(\theta)$ are coefficients determined by the trainable parameters.

This is precisely the structure of a finite trigonometric series! More importantly, since the QNN output is obtained through the expectation value of an observable, it can also be expressed as a finite Fourier series:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} \tag{3.25}$$

Here, Ω denotes the set of accessible frequencies. Unlike classical neural networks with learnable frequencies, such as SIREN, the accessible frequency spectrum of a QNN is primarily determined by its data-encoding strategy and circuit architecture. In particular, both circuit depth, characterized by the number of data-reuploading layers, and circuit width, characterized by the number of qubits involved in data encoding, can influence the accessible frequency spectrum. We will explore these relationships in greater detail in the following sections.

Furthermore, $\omega \in \Omega$ represents an individual frequency component, while $c_\omega(\theta)$ denotes its corresponding trainable Fourier coefficient. From Fourier analysis, we know that the expressive power of a Fourier series—namely, the range of functions it can approximate—depends in part on the richness of its accessible frequency spectrum. Unlike in SIREN, where frequencies can be learned during training, the frequency spectrum of a QNN with a fixed encoding scheme is predetermined by its architecture rather than directly optimized through training.

This naturally raises an intriguing question: **Is a predetermined frequency spectrum a fundamental limitation of QNNs, or could it actually become an advantage?** We will investigate this question in the following sections.

This remarkable connection allows us to study the expressive power of QNNs through their accessible frequency spectra and trainable Fourier coefficients, rather than explicitly tracking the exponentially growing quantum statevector.

3.7 Output Layer: Quantum Measurement

The final stage of a QNN is the measurement layer, which converts quantum information into classical outputs. Unlike classical neural networks, where intermediate activations can be directly accessed, the quantum statesevolving throughout a quantum circuit cannot be directly observed without performing measurements. In particular, a computational-basis measurement of a qubit yields either 0 or 1, with probabilities governed by the Born rule. However, the complex probability amplitudes underlying these outcomes cannot be directly recovered from a single measurement.

How, then, can we extract meaningful numerical outputs from a quantum circuit? A common approach is to measure the expectation value of a quantum observable $\mathcal{O}$.

After the encoding layer and multiple variational rotation and entangling layers, the quantum state can be expressed as

$$|\psi(x,\theta)\rangle = U(x,\theta)\,|0\rangle^{\otimes n} \tag{3.26}$$

where $U(x,\theta)$ represents the overall unitary transformation of the circuit. Despite the potentially complex sequence of quantum operations, the entire ideal circuit can be mathematically represented by a single unitary operator.

Given an observable $\mathcal{O}$, the output of the QNN is defined as its expectation value:

$$\begin{aligned} f_\theta(x) &= \langle\psi(x,\theta)|\,\mathcal{O}\,|\psi(x,\theta)\rangle \\ &= \langle 0|^{\otimes n}\, U^\dagger(x,\theta)\mathcal{O}U(x,\theta)\,|0\rangle^{\otimes n} \end{aligned} \tag{3.27}$$

One of the most commonly used observables in variational QNNs is the Pauli-Z operator:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \tag{3.28}$$

To understand how this measurement works, consider a single qubit in the state

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |\alpha|^2 + |\beta|^2 = 1 \tag{3.29}$$

According to the Born rule, the probabilities of measuring the qubit in states $|0\rangle$ and $|1\rangle$ are

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2$$

Since the eigenvalues of the Pauli-Z operator are $+1$ and -1, its expectation value is

$$\begin{aligned} \langle Z\rangle &= \langle\psi|\,Z\,|\psi\rangle \\ &= |\alpha|^2 - |\beta|^2 \\ &= P(0) - P(1) \\ &= 2P(0) - 1 \end{aligned} \tag{3.30}$$

Consequently, the measurement probabilities can be recovered from the expectation value:

$$P(0) = \frac{1 + \langle Z\rangle}{2}, \qquad P(1) = \frac{1 - \langle Z\rangle}{2} \tag{3.31}$$

Since $-1 \le \langle Z\rangle \le 1$, an expectation value closer to $+1$ indicates a higher probability of measuring $|0\rangle$, whereas a value closer to -1 indicates a higher probability of measuring $|1\rangle$.

Importantly, the expectation value should not be confused with the probability amplitude. While the probability amplitudes α and β are generally complex-valued, the expectation value $\langle Z\rangle$ is real-valued and reflects the difference between the two measurement probabilities.

On real quantum hardware, the expectation value cannot be obtained from a single circuit execution. Instead, the circuit must be executed repeatedly to estimate the expectation value statistically:

$$\widehat{\langle Z\rangle} = \frac{N_0 - N_1}{N} \tag{3.32}$$

where N_0 and N_1 denote the numbers of measurement outcomes 0 and 1, respectively, and $N = N_0 + N_1$ is the total number of measurement shots.

For a multi-qubit system, the same principle applies. Consider the three-qubit QNN introduced earlier. Following the little-endian convention, its final statevector is

$$|\psi\rangle = \begin{bmatrix} a_{000} \\ a_{001} \\ a_{010} \\ a_{011} \\ a_{100} \\ a_{101} \\ a_{110} \\ a_{111} \end{bmatrix} \tag{3.33}$$

where

$$\sum_{i,j,k\in\{0,1\}} |a_{ijk}|^2 = 1 \tag{3.34}$$

To measure the first qubit, q_1, we define the observable

$$\mathcal{O}_1 = I \otimes I \otimes Z \tag{3.35}$$

The corresponding expectation value becomes

$$\begin{aligned} \langle Z_1 \rangle &= \langle \psi | \, \mathcal{O}_1 \, | \psi \rangle \\ &= |a_{000}|^2 - |a_{001}|^2 \\ &\quad + |a_{010}|^2 - |a_{011}|^2 \\ &\quad + |a_{100}|^2 - |a_{101}|^2 \\ &\quad + |a_{110}|^2 - |a_{111}|^2 \end{aligned} \tag{3.36}$$

This expression illustrates that measuring a single qubit generally involves contributions from multiple computational basis states. In particular, the expectation value depends on the marginal measurement probabilities associated with the selected qubit, rather than on any individual probability amplitude.

Similarly, the remaining two qubits can be measured using the observables

$$\mathcal{O}_2 = I \otimes Z \otimes I, \qquad \mathcal{O}_3 = Z \otimes I \otimes I \tag{3.37}$$

By measuring all three qubits, the QNN produces a vector of expectation values:

$$\mathbf{f}_\theta(x) = \begin{bmatrix} \langle Z_1 \rangle \\ \langle Z_2 \rangle \\ \langle Z_3 \rangle \end{bmatrix} \tag{3.38}$$

These expectation values serve as classical output features and can either be used directly for prediction or passed to an additional classical processing layer.

Beyond Pauli-Z measurements, other observables, such as Pauli-X, Pauli-Y, and multi-qubit correlation operators, can also be employed to extract different properties of the quantum state. However, a detailed discussion of these measurement strategies is beyond the scope of this section.

In summary, quantum measurement provides the essential bridge between the quantum representation and the classical output of a QNN. More importantly, expressing the QNN

output as the expectation value of an observable establishes the mathematical foundation for analyzing its function representation, particularly through Fourier analysis.

4 Fourier Analysis of Quanutm Neural Networks and Expressivity

4.1 Basics of Fourier Series and Fourier Transform

Fourier analysis provides a powerful mathematical framework for representing and analyzing functions in terms of their frequency components. The fundamental idea is that a function can be decomposed into a combination of oscillatory basis functions, such as sines, cosines, or complex exponentials. Instead of examining a function solely in its original domain, Fourier analysis allows us to study its behavior in the frequency domain.

Two fundamental tools in Fourier analysis are the *Fourier series* and the *Fourier transform*. Although closely related, they serve different purposes and apply to different classes of functions.

Fourier Series

A Fourier series represents a periodic function as an infinite sum of sinusoidal components with different frequencies, amplitudes, and phases.

For a periodic function $f(x)$ with period T, its Fourier series can be expressed in complex exponential form as

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ik\omega_0 x} \tag{4.1}$$

where $\omega_0 = 2\pi/T$ is the fundamental angular frequency, $k \in \mathbb{Z}$ is the harmonic index, and c_k denotes the Fourier coefficient associated with frequency $k\omega_0$.

Each Fourier coefficient can be calculated by projecting the original function onto its corresponding complex exponential basis:

$$c_k = \frac{1}{T} \int_0^T f(x) e^{-ik\omega_0 x}\, dx \tag{4.2}$$

These coefficients describe the contribution of each frequency component to the original function. In particular, $|c_k|$ determines its magnitude, while the complex phase of c_k determines its phase shift.

According to Euler's formula,

$$e^{i\omega x} = \cos(\omega x) + i \sin(\omega x) \tag{4.3}$$

the complex exponential representation is equivalent to expressing a real-valued periodic function as a sum of sine and cosine functions.

Importantly, different frequencies capture different levels of variation in a function. Low-frequency components describe slowly varying patterns, whereas high-frequency components represent rapid oscillations and fine-grained details.

In practical applications, Fourier series are often approximated using a finite number of frequency components. More generally, a finite trigonometric expansion can be written as

$$f(x) = \sum_{\omega \in \Omega} c_\omega e^{i\omega x} \tag{4.4}$$

where Ω is a finite set of frequencies. Unlike the conventional Fourier series, thefrequencies in this generalized representation are not necessarily integer multiples of a common fundamental frequency.

Fourier Transform

While Fourier series decompose periodic functions into discrete harmonics, the Fourier transform extends frequency-domain analysis to nonperiodic functions.

For a sufficiently well-behaved function $f(x)$, the continuous Fourier transform is defined as

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x}\, dx \tag{4.5}$$

where $\omega \in \mathbb{R}$ represents the angular frequency and $\hat{f}(\omega)$ describes the frequency-domain representation of $f(x)$.

The original function can be reconstructed using the inverse Fourier transform:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x}\, d\omega \tag{4.6}$$

The key distinction is that the Fourier series uses a discrete set of frequencies, whereas the Fourier transform is defined over a continuous frequency variable. Nevertheless, both share the same fundamental principle: a function can be understood through its frequency components and their corresponding coefficients.

From a function-approximation perspective, the frequency spectrum reveals which oscillatory patterns are present in a function. This perspective is particularly useful when studying the expressive power of machine learning models.

4.2 Connection to Quantum Neural Networks

The significance of Fourier analysis becomes apparent when examining the mathematical representation of Quantum Neural Networks (QNNs).

As discussed previously, the output of a variational QNN is typically defined through the expectation value of an observable:

$$f_\theta(x) = \langle 0 | U^\dagger(x, \theta) \mathcal{O} U(x, \theta) | 0 \rangle \tag{4.7}$$

For the finite-dimensional quantum circuits with data-encoding operations considered in this paper, this expectation-value function can be expressed as a finite Fourier series:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} \tag{4.8}$$

Here, Ω denotes the set of accessible frequencies, while $c_\omega(\theta)$ represents the Fourier coefficients controlled by the trainable circuit parameters.

Unlike conventional Fourier approximations, where the selected frequencies may be chosen or optimized for a particular task, the accessible frequencies of a QNN with a fixed data-encoding scheme are determined by its encoding operations and circuit architecture. The trainable parameters primarily control the corresponding Fourier coefficients, subject to constraints imposed by the circuit.

Therefore, Fourier analysis provides a natural framework for investigating two fundamental aspects of QNN expressivity: which frequencies a quantum circuit can represent, and how flexibly it can adjust their coefficients.

In the following section, we will examine how these properties emerge from quantum data encoding and how circuit depth and width influence the accessible frequency spectrum.

4.3 Quantum Neural Network as a Finite Fourer Series

4.4 Data-Reuploading and accessible frequencies

4.5 Expressivity